```
    return bookRepository.findByIsbn(isbn)                    ❸
      .orElseThrow(() -> new BookNotFoundException(isbn));
  }

  public Book addBookToCatalog(Book book) {
    if (bookRepository.existsByIsbn(book.isbn())) {           ❹
     throw new BookAlreadyExistsException(book.isbn());
    }
    return bookRepository.save(book);
  }

  public void removeBookFromCatalog(String isbn) {
    bookRepository.deleteByIsbn(isbn);
  }

  public Book editBookDetails(String isbn, Book book) {
    return bookRepository.findByIsbn(isbn)
      .map(existingBook -> {
        var bookToUpdate = new Book(                          ❺
          existingBook.isbn(),
          book.title(),
          book.author(),
          book.price());
        return bookRepository.save(bookToUpdate);
      })
      .orElseGet(() -> addBookToCatalog(book));               ❻
  }
}
```

❶ Stereotype annotation that marks a class to be a service managed by Spring

❷ BookRepository is provided through constructor autowiring.

❸ When trying to view a book that doesn't exist, a dedicated exception is thrown.

❹ When adding the same book to the catalog multiple times, a dedicated exception is thrown.

❺ When editing the book, all the Book fields can be updated except the ISBN code, because it's the entity identifier.

❻ When changing the details for a book not in the catalog yet, create a new book.

NOTE The Spring Framework provides two flavors of dependency injection: *constructor-based* and *setter-based*. We'll use constructor-based dependency injection in any production code, as advocated by the Spring team, because it ensures that the required dependencies are always returned fully initialized and never `null`. Furthermore, it encourages building immutable objects and improves their testability. For more information, refer to the Spring Framework documentation (https://spring.io/projects/spring-framework).